Vivekanand Education Society's Institute of Technology

Department of Computer Engineering

Group No.: 19
Date :- 11/08/2025

Project Synopsis (2025-26) - Sem VII

DeBug - A Decentralised Bug-Bounty Platform

Ms. Geocey Shejy
Professor, CMPN Department

Rohit Shahi
V.E.S.I.T
2022.rohit.shahi@ves.ac.in

Gopal Vanjarani
V.E.S.I.T
2022.gopal.vanjarani@ves.ac.in

Gaurav Mahadeshwar
V.E.S.I.T
2022.gaurav.mahadeshwar@ves.ac.in

Umesh Tolani
V.E.S.I.T
2022.umesh.tolani@ves.ac.in

Abstract :

Traditional bug bounty platforms centralize custody of funds and decision-making, leading to trust gaps, opaque payouts, and limited researcher privacy. This project proposes a hybrid Web3 marketplace where bounty owners post security challenges, escrow rewards on a low-cost Layer-2 blockchain, and researchers submit encrypted, private reports stored off-chain (IPFS/Arweave). Smart contracts enforce escrow, acceptance, and verifiable payouts; the off-chain layer handles authentication, submission workflows, reputation, and notifications. Optional zero-knowledge (ZK) identity supports anonymous yet reputable disclosures. The system aims to deliver predictable payouts, auditable decisions, and reduced operational friction. We detail the architecture, methodology, components, evaluation plan, and security controls for an academically rigorous, reproducible implementation.

Introduction :

Modern software organizations rely on external security researchers for continuous vulnerability discovery. Centralized bounty platforms solve coordination but introduce issues: (1) projects must trust a third party to hold and release funds, (2) researchers have limited privacy during triage, and (3) reputation is siloed and non-portable. With the maturity of smart contracts, decentralized identity, and content-addressed storage, we can separate trust-critical money flows (on-chain) from user-experience-critical workflows (off-chain). This split yields verifiable funding and payouts without sacrificing usability.

Key drivers:

- **Trust & Transparency**: On-chain escrows, immutable events for listing creation, acceptance, and payout.
- **Privacy-Preserving Disclosure**: Client-side encryption of reports; private triage before public disclosure.
- **Cost & Latency**: L2 chains minimize gas while keeping finality fast enough for marketplace UX.
- **Reputation Portability**: Researcher reputation stored off-chain but anchored via signed attestations; optional ZK identity prevents deanonymization while proving uniqueness.

Problem Statement :

Despite widespread adoption, today's bounty ecosystems face persistent trust and efficiency gaps:

- Payout Frictions: Delays or denials, opaque severity decisions, and custodial risk undermine researcher confidence.
- Privacy & IP Risks: Sensitive proofs-of-concept can leak or be misused during triage.
- Fragmented Reputation: Researcher credibility remains platform-locked, with little portability across programs.
- Operational Overhead: Manual communications, weak audit trails, and unverifiable state changes hinder scalability.
- Rigid On-Chain Designs: All-or-nothing architectures push excessive state on-chain, driving up gas costs and complexity.

Addressing these challenges requires a marketplace that combines verifiable funding and automated payouts with a private, streamlined submission process. By enabling portable reputation, reducing operational friction, and ensuring measurable, auditable security outcomes, such a system can bridge the trust gap between researchers and projects while maintaining scalability and privacy.

Proposed Solution :

A **hybrid architecture** with minimal, security-critical logic on-chain and all high-volume UX off-chain:

On-chain (L2, Solidity) :

- Bounty escrow creation and top-ups (native token or ERC-20 like USDC).
- Acceptance & payout functions (full/partial releases; split payouts).
- Dispute entry points with time-locks and multi-sig/DAO arbitration hooks.
- Events for state changes consumed by off-chain indexers.

Off-chain (Next.js + Node/Supabase) :

- Auth, role management, listings metadata, submissions, arbitration records.
- Client-side encryption of vulnerability reports; storage on IPFS/Arweave (CIDs only in DB).
- Reputation system (accepted findings, severity-weighted) with signed attestations.
- Notifications (email/in-app) and audit logs.
- Worker services for on-chain event sync, IPFS pinning, and anti-abuse rate limiting.

Privacy & Identity :

- Wallet-based login; optional ZK identity (e.g., Semaphore-style) for anonymous submissions with Sybil resistance.
- PGP or ephemeral key exchange for end-to-end encrypted disclosure.

Hardware , Software and tools Requirements :

Hardware (Dev/Test) -

- x64 machine with 8–16 GB RAM, 50 GB storage.
- Optional: local IPFS node (Docker) for faster dev.

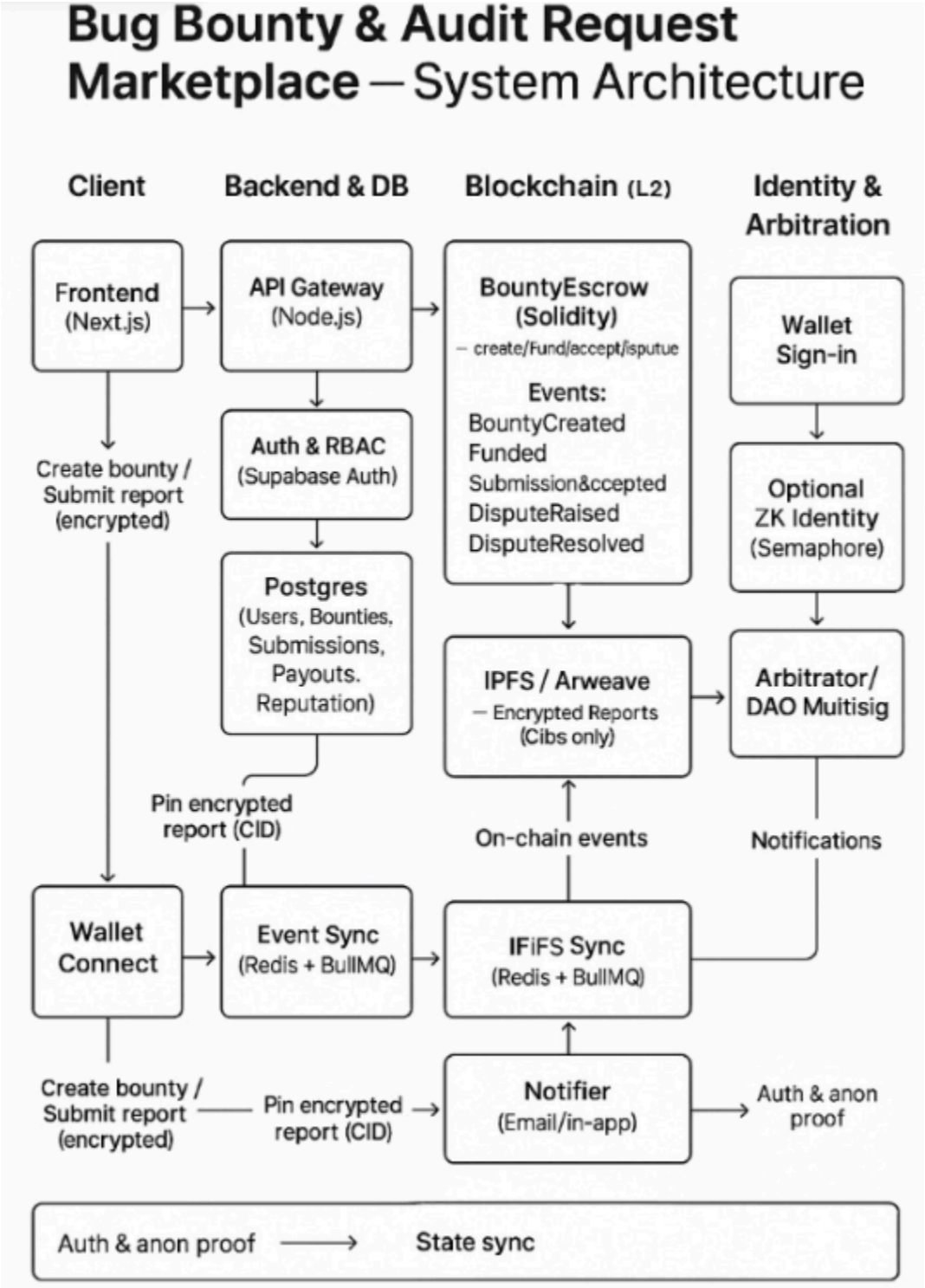
Software Stack -

- **Frontend:** Next.js (React), Tailwind CSS.
- **Backend:** Node.js (Express/Fastify) or Next API routes; Supabase (Auth + Postgres).
- **Blockchain:** Solidity, Hardhat/Foundry; deploy on zkSync/Polygon/Arbitrum testnets then mainnet L2.
- **Storage:** IPFS (Pinata/Infura); optional Arweave for long-term permanence.
- **Identity:** WalletConnect/MetaMask; optional Semaphore-style ZK.
- **Workers & Queues:** Redis + BullMQ for event processing and pinning jobs.
- **Security Tooling:** OpenZeppelin libraries; eslint/prettier; SAST/dep audit; secret scanning.
- **CI/CD:** GitHub Actions; Vercel for frontend; Render/Fly.io for API.

Utilities -

- PGP tooling for client-side crypto, file scanners for malware, monitoring (Grafana/Prometheus or hosted APM), log aggregation.

Block Diagram :



Methodology :

End-to-End Flow

1. **Bounty Creation:** Owner drafts bounty, sets tiers/deadline; funds escrow via contract.
2. **Submission:** Researcher encrypts report client-side; uploads to IPFS; metadata stored off-chain.
3. **Triage & Decision:** Owner reviews privately; may request clarification. Acceptance triggers on-chain payout.
4. **Dispute (Optional):** If contested, invoke arbitration (off-chain review + on-chain release via arbiter key/DAO).
5. **Settlement & Reputation:** Contract emits payout event; indexer updates DB, researcher reputation increments; notifications sent.

System Design :

Smart Contract Layer (Solidity, L2) -

- **Core contract:** BountyEscrow
    - createBounty(id, token, amount, metaHash)
    - fundBounty(id, amount)
    - acceptSubmission(id, researcher, amount) (supports split payouts)
    - raiseDispute(id, submissionId) → pauses release window
    - resolveDispute(id, decision) by arbitrator key/DAO
    - Events: BountyCreated, BountyFunded, SubmissionAccepted, DisputeRaised, DisputeResolved
- **Controls:** Reentrancy guards, access modifiers, pausability, timelocks, SafeERC20, pull-payment pattern where possible.

Off-Chain Services -

- **API Gateway:** Auth (JWT/Supabase), input validation, rate limits, anti-automation (CAPTCHA), abuse-report endpoints.
- **Indexer/Workers:** Listen to contract events, reconcile DB state, send notifications, recalc reputation, re-pin IPFS.
- **Storage:** Postgres for relational data; IPFS/Arweave for encrypted blobs; hashed pointers stored in DB.
- **Encryption:** PGP public key of bounty owner; or ECDH ephemeral symmetric key exchange; AES-GCM payloads.

Data Model (key tables) -

- users(id, wallet, display_name, is_verified, reputation, zk_identifier)
- projects(id, owner_id, name, repo_url, contact)
- bounties(id, project_id, title, description_cid, token, reward_total, tiers_json, status, escrow_tx, deadline)
- submissions(id, bounty_id, researcher_id, anon_id, report_cid, severity_claimed, status, created_at)
- payouts(id, submission_id, amount, token, tx_hash, status)
- arbitrations(id, submission_id, arbitrator_id, decision, notes, tx_hash)
- reputation_events(id, user_id, delta, reason, created_at)

Proposed Evaluation Measures :

Category	**Metric**	**Target / Rationale**
Functional Correctness	End-to-end success rate (create→payout)	≥ 95% on test suite
	Event→DB sync latency	< 3s median on L2
Security (On-Chain)	Reentrancy & access control tests	100% pass
	Unit test coverage (contracts)	≥ 90% lines/branches
Privacy	Zero plaintext report leaks	0 incidents in tests
Cost	Gas per create/fund/payout	Benchmarked on chosen L2
Performance	API p95 latency under load	< 300 ms @ 200 RPS
Trust & UX	Payout confirmation time (owner accept→receipt)	< 60 s median
Reputation Robustness	Sybil resistance checks	No duplicate identities over test set

Conclusion :

The proposed marketplace splits trust and usability concerns cleanly: money on-chain, workflow off-chain. It enables verifiable, time-bounded payouts; private, encrypted triage; and portable reputation. Built on L2 networks and open-source tooling, the system is cost-efficient and scalable. The methodology provides a realistic semester-length path to a deployable MVP, with clear success metrics and room for advanced features like ZK identity and DAO-based arbitration.

References :

[1] G. Wood, Ethereum: A Secure Decentralised Generalised Transaction Ledger (Yellow Paper). [Online]. Available: https://ethereum.github.io/yellowpaper/paper.pdf

[2] OpenZeppelin, OpenZeppelin Contracts: Secure Smart-Contract Library and Patterns. [Online]. Available: https://docs.openzeppelin.com/contracts

[3] F. Vogelsteller and V. Buterin, "ERC-20 Token Standard," Ethereum Improvement Proposal 20, Nov. 2015. [Online]. Available: https://eips.ethereum.org/EIPS/eip-20

[4] D. T. C. O'Hara et al., "EIP-2612: Permit Extension for ERC-20 Tokens," Ethereum Improvement Proposal 2612, Jan. 2020. [Online]. Available: https://eips.ethereum.org/EIPS/eip-2612

[5] Protocol Labs, IPFS Documentation: Content Addressing and Pinning Best Practices. [Online]. Available: https://docs.ipfs.tech

[6] Arweave, Arweave Documentation: Permanent Storage and Gateways. [Online]. Available: https://docs.arweave.org

[7] P. Feichtinger et al., Semaphore: Privacy-Preserving Identity Protocol. [Online]. Available: https://semaphore.appliedzkp.org

[8] OWASP Foundation, OWASP Vulnerability Disclosure and Severity (CVSS) Guidelines. [Online]. Available: https://owasp.org/www-project-vulnerability-disclosure

[9] Polygon Technology, Polygon Official Documentation. [Online]. Available: https://docs.polygon.technology

[10] Matter Labs, zkSync Documentation. [Online]. Available: https://docs.zksync.io

[11] Offchain Labs, Arbitrum Official Documentation. [Online]. Available: https://developer.arbitrum.io